Vidush Somany Institute of Technology and Research, Kadi

CT405-N: Object Oriented Programming Using Java

Laboratory Assignment #1

1.1 (a) Write a Java program to find the factorial of a number. Initialize the number in the program itself. (`E.g. int n=5;`)
Solution: https://github.com/hbpatel1976/Java/blob/main/Assignment11a.java

1.1 (b) Write a Java program to find the factorial of a number. Read the number from the command line argument.

```
Example:
C:\>javac factorial.java
C:\>java factorial 5
Answer=120
```

Solution: https://github.com/hbpatel1976/Java/blob/main/Assignment11b.java

1.1 (c) Write a Java program to find factorial of a number. Read the number from the keyboard while running the program.

```
Example:
C:\>javac factorial.java
C:\>java factorial
Enter the number for which you wish to find factorial: 5
Answer=120
```

Solution: https://github.com/hbpatel1976/Java/blob/main/Assignment11c.java

1.2 Following is a sample income tax slab.
Up to Rs.2.5 lakh - No Tax (NIL)
Rs.2.5 lakh - Rs.5 lakh - 5%
Rs.5 lakh - Rs.7.5 lakh - 10%
Rs.7.5 lakh - Rs.10 lakh - 15%
Rs.10 lakh - Rs.12.5 lakh - 20%
Rs.12.5 lakh - Rs.15 lakh - 25%
Above Rs.15 lakh - 30%
Write a Java program which initialized the earning of an employee (E.g. `int income=1950000;`). Your program should display the outcome (E.g. `Income tax to be paid: 3,22,500`)
Solution: https://github.com/hbpatel1976/Java/blob/main/Assignment12.java

1.3 Design a `class Bank` with `accountNo`, `name` and `balance` as three members. The class should have methods viz. (a) `depositMoney`, (b) `withdrawMoney` and (c) `askBalanace`. Write a Java program to create an object of `Bank` class in main function and invoke all the three functions from main to test their working.
Solution: https://github.com/hbpatel1976/Java/blob/main/Assignment13.java

1.4 Create a `class stack` with `data[size]` and `stackTop` as data members and `push`, `pop` and `display` as methods. Write a Java program to create an object of stack class in main and invoke all the three functions from main to test their working.

1.5 Write a Java program to display all the prime numbers between two given numbers. (You may initialize the two numbers **or** read them from keyboard **or** input them as command line arguments). You may also try,

- Write a program to check whether the given number is Palindrome or not. A palindrome number reads the same forwards and backward, like 121, 4554, or 9, where reversing the digits doesn't change the original number.

Vidush Somany Institute of Technology and Research, Kadi

CT405-N: Object Oriented Programming Using Java

- The Fibonacci series is a sequence of numbers where each number is the sum of the two preceding ones, starting with 0 and 1: 0, 1, 1, 2, 3, 5, 8, 13, 21..., continuing infinitely. Generate Fibonacci series for 'n' terms.

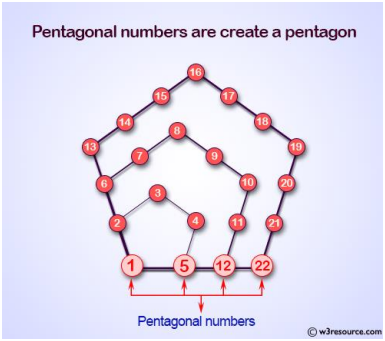
Vidush Somany Institute of Technology and Research, Kadi

CT405-N: Object Oriented Programming Using Java

Laboratory Assignment #2

2.1 The formula for calculating Body Mass Index (BMI) is weight (kg) / [height (m)]^2.
A person is "Underweight" if BMI is below 18.5, "Normal" if BMI is between 18.5 to 24.9, "Overweight" if BMI is between 25.0 to 29.9, and "Obese" if BMI is 30.0 and Above.
Write a Java program that reads "weight" (in kg) and "height" (in meter) and display the status of the person (e.g. "Underweight", "Normal" etc.)

2.2 Following figure shows the pentagonal numbers creation for a typical pentagon. (Source: w3resource.com)



For any number "i", the pentagonal number is computed using the equation [i*(3*i-1)]/2. Hence, write a Java program with *getPentagonalNumber* as method that takes a number "i" as input and returns the equivalent pentagonal number. Print first 4 (n=4) pentagonal numbers (E.g. 1, 5, 12, 22 as shown in the figure).

2.3 An employee gets paid (hours worked) × (basic pay), for each hour up to 40 hours for a week. For every hour over 40, they get overtime = (base pay) × 1.5. The base pay must not be less than the minimum wage (Rs. 150 an hour). If it is, print an error. If the number of hours is greater than 60 per, print an error message. Write a Java program that reads (1) basic pay per hour and (2) number of hours worked per week and display the wage/salary to be paid to him/her.

2.4 Write a Java program to implement Vector with following functionality.
Create a vector *student* with syntax: `Vector student = new Vector();`
Using the syntax `student.insertElementAt("Mobile",0);` to add "*Mobile*" at 0th location OR use the syntax `Student.addElement("Mobile");`
Subsequently, add "*Laptop*", "*Watch*", "*Shoes*", "*T-shirt*" and "*Jeans*".
Now, display all the items in student vector using `System.out.println(student);`
Later, the *student* decided to return "Jeans" using `student.remove("Jeans");`
After removing *Jeans*, display the items available with the *student*.
(You may try other functions related to Vectors mentioned in the lecture note)

2.5 Create a class Car with three data members [model, color, brand] and a method [setData, displayCarInfo]. Create following objects.
Car 1: Model-City, Color-White, Brand-Honda
Car 2: Model-Desire, Color-Red, Brand-Maruti
Car 3: Model-XUV500, Color-Maroon, Brand-Mahindra
You may use *setData* method to pass all these values. For example, `car1.setData(String mdl, String clr, String brnd)`. Use the displayCarInfo method to print the details of respective car (E.g. `car1.displayCarInfo();`)

Vidush Somany Institute of Technology and Research, Kadi

CT405-N: Object Oriented Programming Using Java

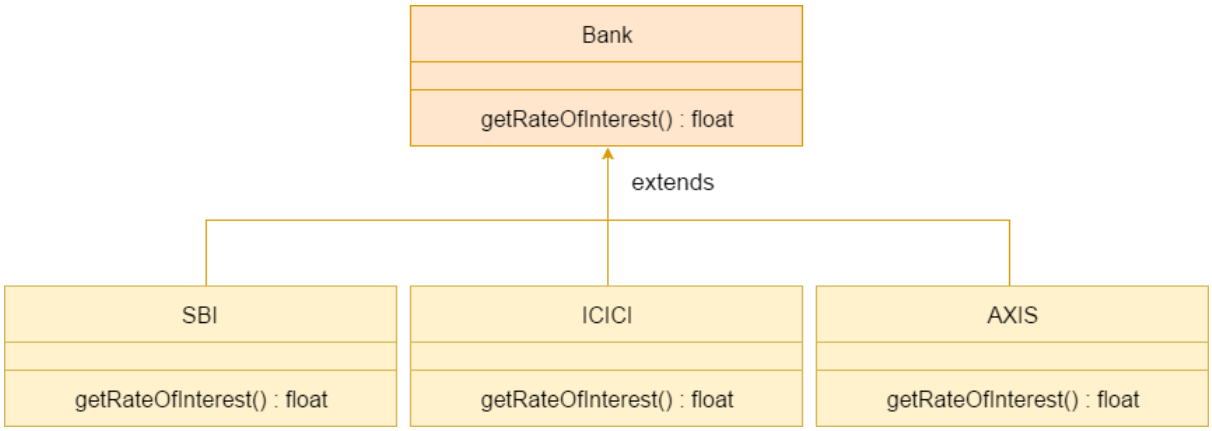
Laboratory Assignment #3

3.1 Define a class ***Doctor*** with Name as data member. Inherit a class ***Surgeon*** from the Doctor class with an extra data member Specialization and Consulting Charge per hour. Inherit another class ***Physician*** with an extra members Speciality and Fees per hour. Define objects **S1** (Specialization= Gynaecologist, Fees per hour= Rs. 2000) and **S2**(Specialization= Paediatrician, Fees per hour= Rs. 1500). Define objects **P1** (Speciality=Heart, Consulting Charge per hour= Rs. 4000) and **P2**(Speciality=Orthopaedic, Consulting Charge per hour = Rs. 2500). Write a main method to create objects as mentioned and required methods to read/display the information. NOTE: Sub-class must call super class constructor.

Solution: https://github.com/hbpatel1976/Java/blob/main/Medical.java

3.2 Create a superclass ***Animal*** that has a method called animalSound(). Subclasses of ***Animal*** could be ***Pig***, ***Cat***, and ***Dog***. And they also have their own implementation of an animal sound (e.g.: the pig oinks, and the cat meows, dog barks etc.) Now create objects of ***Pig*** and ***Dog*** objects and call the animalSound() method on both of them. [Inheritance and method overriding]

3.3 Consider a scenario where ***Bank*** is a class that provides functionality to get the rate of interest. However, the rate of interest varies according to banks. For example, ***SBI***, ***ICICI*** and ***AXIS*** banks could provide 7%, 7.5%, and 7.75%.



Write a Java Program to demonstrate the real scenario of Java Method Overriding where three classes (***SBI***, ***ICICI*** and ***AXIS***) are overriding the method of a parent class (***Bank***)

Vidush Somany Institute of Technology and Research, Kadi

CT405-N: Object Oriented Programming Using Java

Laboratory Assignment #4

4.1 (Final method) Define a class ***Result*** with *dbms*, *os* and *java* as data members. Write a constructor with three arguments that initializes the three data members with these three arguments. Define a `final` method `calculate()` that returns the total of all the three subjects. In the main method, create an object of class ***Result*** and invoke the method `calculate()`.

Solution: https://github.com/hbpatel1976/Java/blob/main/AbstractClassBird.java

4.2 (Abstract class) Define an abstract class **Animal** with three data members viz. `vegetarian` (boolean), `runningSpeed` (int), `animalName` (String). The class **Animal** should have a constructor with three arguments which are assigned to the data members mentioned above. The class **Animal** should also have an abstract method `display()`. Now, inherit two classes **Carnivorous** and **Vegetarian** from **Animal** and write appropriate constructor and the display method. In the main method, create an object Lion of class **Carnivorous** and Cow of class **Vegetarian** with appropriate data members. The Lion and the Cow object should call the `display()` function to print appropriate messages.

Solution: https://github.com/hbpatel1976/Java/blob/main/FinalMethodAssignment1.java

Vidush Somany Institute of Technology and Research, Kadi

CT405-N: Object Oriented Programming Using Java

Laboratory Assignment #5

5.1 GENERATE Output / Error (if any) with REASONING

5.1.1. Generate Output / Error (if any) for the following code. Justify your answer.

```
class Main
{
  public static void main(String[] args)
{
    // create a final variable
    final int AGE = 32;
      AGE = 45;   // try to change the final variable
    System.out.println("Age: " + AGE);
  }
}
```

5.1.2. Generate Output / Error (if any) for the following code.

```
abstract class Shape
{
      abstract void draw();
}
class Rectangle extends Shape
{
      void draw(){System.out.println("drawing rectangle");}
}
class Circle1 extends Shape
{
      void draw(){System.out.println("drawing circle");}
}
class TestAbstraction1
{
      public static void main(String args[])
      {
            Circle1 s=new Circle1();
            s.draw();
      }
}
```

5.1.3. What will be the correct sequence of the Output / Error (if any) for the following code?

```
class Example1
{
public static void main(String args[])
      {
      try
            {
              System.out.println("First statement of try block");
              int num=45/3;
              System.out.println(num);
            }
      catch(ArrayIndexOutOfBoundsException e)
            {System.out.println("ArrayIndexOutOfBoundsException");}
      finally
            {System.out.println("finally block");}
      System.out.println("Out of try-catch-finally block");
      }
}
```

(A) finally block, First statement of try block, 15, Out of try-catch-finally block
(B) finally block, First statement of try block, Out of try-catch-finally block,15
(C) First statement of try block, 15, finally block, Out of try-catch-finally block
(D) First statement of try block, finally block, 15, Out of try-catch-finally block

5.1.4. Generate the output of the following code. Give supporting arguments/ reasons for your Output / Error (if any).

```
class Demo
{
   final int MAX_VALUE=99;
   void myMethod()
      {
            MAX_VALUE=101;
      }
   public static void main(String args[])
      {
      Demo obj=new  Demo();
      obj.myMethod();
      }
}
```

5.1.5. Generate the output of the following code. Give supporting arguments/ reasons for your Output / Error (if any).

```
public class FinalMethodExample
{
   public final void display()
      {System.out.println("We are implementing Final method example");}
   public static void main(String args[])
      {new FinalMethodExample().display();}
   class Sample extends FinalMethodExample
      {
            public void display()
      {System.out.println("hi");}
      }
}
```

5.2 [Parameterized constructor] Create a class Student with two data members viz. id and name. Create a parameterized constructor with two arguments for id and name. Create a method display() to display id and name of the two students. Create two objects s1 and s2 for the same.

5.3 [Constructor Overloading] Create a class Person with three data members name, id and age. Create a constructor with two arguments for id and name and second constructor with three arguments for id , name and age. Create two objects p1 for the first constructor and p2 for the second constructor. Create a display() method to display id, name and age of the person.

5.4 [Abstract class and abstract method]: Create MyClass as the abstract super class with an abstract method calculate(). This method does not have any body within it. Sub1, Sub2 and Sub3 are three sub classes where the abstract method is implemented as per the requirements of the objects. Sub1 calculates square root as x*x, Sub2 calculates square root using Math.sqrt(x) and Sub3 calculates cube of x. Since the same abstract method is implemented differently for different objects , they can perform different tasks.

Write a program where the abstract class MyClass has one abstract method which has got various implementations in sub classes.

Vidush Somany Institute of Technology and Research, Kadi

CT405-N: Object Oriented Programming Using Java

Laboratory Assignment #6

6.1 (Abstract method / class) Declare an abstract class **shape** having `radius`, `length` and `width` as data members and `showArea` as an abstract method. Derive a class **rectangle** from shape with a constructor accepting two integer arguments which are to be copied into `length` and `width` respectively. Override `showArea` method in **rectangle**. Also derive another class **circle** from **shape** with a constructor accepting one integer argument which is to be copied into `radius`. Override `showArea` method in **circle**. In main method, declare objects of classes **rectangle** and **circle**. Also, call `showArea` method of respective classes to display the area.

Solution: https://github.com/hbpatel1976/Java/blob/main/AbstractClassShape.java

6.2 (Multiple Inheritance) Declare an `interface` **shape** having a `final static` variable PI=3.14. Declare a method `computeArea` in shape. Now, inherit a class **rectangle** from **shape** and override the function `computeArea` to print area of a rectangle. Further, inherit a class **circle** from **shape** and override the function `computeArea` to print area of a circle. In main method, create objects of **rectangle** and **circle** and invoke the `computArea` methods of respective class.

Solution: Source: https://github.com/hbpatel1976/Java/blob/main/InterfaceTest.java

6.3 (Multiple Inheritance) Declare an `interface` **motorcycle** with `speed` as a data member and `distanceCovered` as a method. Declare another `interface` **cycle** with `time` as a data member and `runningTime` as a method. Now, derive a class **TwoWheeler** from the `interface` **motorcycle** and **cycle** and override both the methods of the interfaces here in this class to display distance travelled (speed x time) and running time, respectively. Create an object of the class **TwoWheeler** in main method and invoke the methods to display appropriate messages.

Solution: https://github.com/hbpatel1976/Java/blob/main/MultipleInheritance1.java

6.4 (Multiple Inheritance) Create a class **student** with `rollNumber` as data member and two methods viz. `getNumber` and `putNumber` to read and display the `rollNumber`, respectively. Derive a class **Test** from **student** class having two data members, `part1` and `part2`. Add two methods, `getMarks` to read/set marks of `part1` and `part2` and `putMarks` to display them. Create an interface **Sports** with `sportIndex` as data member and a method `putIndex` to display the same. Now, create a class **Results** which is derived (extended) from the class **Test** and (implemented) from the interface **Sports** with `total` as data member. The method `display` in **Results** class should display the total of `part1`, `part2` and `sportIndex`. Create object of **Results** class in main method and test the different methods by invoking them.

Solution: https://github.com/hbpatel1976/Java/blob/main/Hybrid.java

Vidush Somany Institute of Technology and Research, Kadi

CT405-N: Object Oriented Programming Using Java

Laboratory Assignment #7

7.1 Number Guessing Game:

The user would be asked to guess a number (n) between a range of numbers (n1, n2). The use would be given fixed number of attempts (a). The outcome would be "Yes, you guessed it right!" if the use predicts the number correct in the given attempts OR "Sorry, better luck next time" if s/he can't. Needless to mention that the number to be guessed should be randomly generated each time the program is run.

7.2 Password Validation:

The user would be asked to enter a password which needs to follow certain conditions.

The length of password must be between 8 characters to 20 characters. It must contain minimum 1 upper case letter, 1 lower case letter, 1 digit, 1 special character. Upon failing one of these conditions, the system should prompt specific error response.

If the use wants, the system should generate a random password for him/her with all the conditions mentioned above.

7.3 Shopping Cart:

This project should include file management. Product file containing Product ID, Product Name, Product Price, Discount, Quantity. The use will have following menu.

- Select Product
- Remove Product
- Update Quantity
- Empty Cart
- Finalize Cart & Generate Bill

7.4 Elective Course Offering:

This project should include file management. There should be one file containing Course Code, Course Name, Enrolled Students. Other file should contain all the students' details such as Enrolment Number, Name and Elected Course Opted. A course cannot be allotted to more than n students. The students have choice to view the Elective Course Detail along with the number of students who opted for them. The student can then make a choice for his/her elective.

7.5 Currency Converter:

User may statically initialize few currencies (E.g. INR, USD, EURO, British Pound, Chinese Yuan etc) along with their exchange rate. The user would first be asked to enter an amount, these select from_currency and to_currency. The system should display equivalent conversion.

Vidush Somany Institute of Technology and Research, Kadi

CT405-N: Object Oriented Programming Using Java

Laboratory Assignment #8

8.1 A calculator application performs division of two numbers entered by the user. Sometimes, the user may enter zero as the divisor, which causes the program to terminate abnormally. Write a Java program to: (a) Accept two integer values from the user, (b) Perform division operation, (c) Handle division by zero using `try-catch`, (d) Display an appropriate error message. Ensure normal program termination under any value.

8.2 A program stores marks of students in an array. Users may attempt to access an element using an invalid index value. Write a Java program to: (a) Store integer values in an array, (b) Accept an index value from the user, (c) Display the element at the given index, (d) Handle `ArrayIndexOutOfBoundsException`

8.3 A user enters numeric data in the form of text, which may not always be a valid number. Write a Java program to: (a) Accept input as a string, (b) Convert the string into an integer, (c) Handle invalid input using `NumberFormatException`

8.4 A program performs arithmetic operations on array elements, where multiple types of runtime errors may occur. Write a Java program to: (a) Perform division on array elements, (b) Handle multiple exceptions using multiple catch blocks, (c) Demonstrate handling of: (i) `ArithmeticException` (ii) `ArrayIndexOutOfBoundsException`

8.5 A file-processing program must release system resources irrespective of whether an exception occurs or not. Write a Java program to (a) Demonstrate the use of `finally` block, (b) Show that the `finally` block executes in all situations, (c) Display appropriate messages for execution flow

8.6 A banking application allows withdrawal from an account. If the withdrawal amount is greater than the available balance, an exception should occur. Write a Java program to: (a) Accept account balance and withdrawal amount, (b) Perform withdrawal operation, (c) Handle runtime exception for insufficient balance, (d) Display appropriate error message.

Vidush Somany Institute of Technology and Research, Kadi

CT405-N: Object Oriented Programming Using Java

Laboratory Assignment #9

9.1 A system needs to demonstrate how a thread moves through different states such as creation, execution, waiting, and termination during program execution. Write a Java program to: (a) Create a thread using the `Thread` class, (b) Demonstrate different thread states, (c) Use thread methods such as `start`(), `sleep`(), and `join`(), (d) Display messages indicating the life cycle of the thread

9.2 An application needs to execute multiple independent tasks concurrently without extending the `Thread` class, to allow better design flexibility. Write a Java program to: (a) Create threads using the `Runnable` interface, (b) Implement the `run`() method, (c) Execute multiple tasks simultaneously, (d) Display thread names during execution

9.3 A system is required to perform two tasks at the same time: one task prints even numbers, while another prints odd numbers. Write a Java program to: (a) Create two threads, (b) Assign separate tasks to each thread, (c) Execute both threads concurrently, (d) Demonstrate parallel execution using thread scheduling

9.4 A banking system allows multiple users to access the same account simultaneously. Unsynchronized access may lead to incorrect balance updates. Write a Java program to: (a) Create a shared `BankAccount` object, (b) Allow multiple threads to perform withdrawal operations, (c) Use synchronized methods or blocks, (d) Prevent race conditions and ensure data consistency

9.5 A producer generates data and a consumer processes it. Both share a common resource, and improper synchronization may cause inconsistent behaviour.
Write a Java program to: (a) Implement producer and consumer threads, (b) Use thread synchronization techniques, (c) Control thread execution using `wait`() and `notify`(), (d) Ensure proper coordination between threads

Vidush Somany Institute of Technology and Research, Kadi

CT405-N: Object Oriented Programming Using Java

Laboratory Assignment #10

10.1 A college administration system needs to store textual information such as notices or student details in a text file and retrieve it whenever required. Write a Java program to: (a) Accept text input from the user, (b) Write the data into a text file, (c) Read the contents of the file and display it on the screen.
Use appropriate character I/O classes for file operations.

10.2 A system utility is required to copy the contents of one file to another without modifying the data, such as copying an image or executable file.
Write a Java program to: (a) Read data from a binary file, (b) Write the data into another binary file, (c) Preserve the original file content
Use appropriate binary I/O classes.

10.3 An application needs to store complete objects permanently and retrieve them later for processing or display. Write a Java program to: (a) Create a class (e.g., Student) that implements `Serializable`, (b) Store object data into a file, (c) Retrieve and display the object from the file.
Demonstrate object I/O (serialization and deserialization).

10.4 An organization needs to access employee records directly without reading the entire file, enabling faster updates and retrieval. Write a Java program to: (a) Store employee records in a file, (b) Use `RandomAccessFile` to read and write data, (c) Access a specific record based on file pointer position

10.5 A file-processing application must handle errors such as missing files or input/output failures without crashing. Write a Java program to: (a) Perform file read and write operations, (b) Handle I/O-related exceptions, (c) Display meaningful error messages.
Demonstrate safe file handling in Java.

Vidush Somany Institute of Technology and Research, Kadi

CT405-N: Object Oriented Programming Using Java

Laboratory Assignment #11

11.1 A desktop application requires a form to collect user details such as name, gender, and qualification using graphical components. Write a Java program to: Create a GUI using `Frame` (or `JFrame`). Use GUI components such as: (a) Labels, (b) Text Fields, (c) Radio Buttons, (d) Check Boxes, (e) Buttons. Arrange components using an appropriate layout manager.
Display entered user details when a button is clicked Demonstrate GUI basics and action event handling.

11.2 A GUI designer needs to understand how different layout managers arrange components within a window. Write a Java program to: Create a window using `Frame`. Demonstrate the following layout managers: (a) Flow Layout, (b) Border Layout, (c) Grid Layout. Add multiple buttons or labels to show component positioning. Display the effect of each layout clearly

11.3 A graphical application must respond to user interactions such as mouse clicks and key presses. Write a Java program to: (a) Handle mouse events (mouse click, mouse movement), (b) Handle keyboard events (key pressed, key released), (c) Display event details such as key code or mouse coordinates. Use appropriate event listener interfaces

11.4 A desktop application requires a menu system to perform common operations such as changing background color or exiting the application. Write a Java program to: (a) Create a window with a menu bar, (b) Add menus and menu items, (c) Handle menu selection events.
Perform actions such as: (i) Changing window properties, (ii) Displaying a dialog box, (iii) Exiting the application.

11.5 An instructor wants to demonstrate the working and life cycle of a Java applet.
Write a Java applet program to: Implement applet life cycle methods: (a) `init()`, (b) `start()`, (c) `paint()`, (d) `stop()`, (e) `destroy()`.
Display messages indicating the execution of each life cycle method. Embed the applet in an HTML file and execute it.